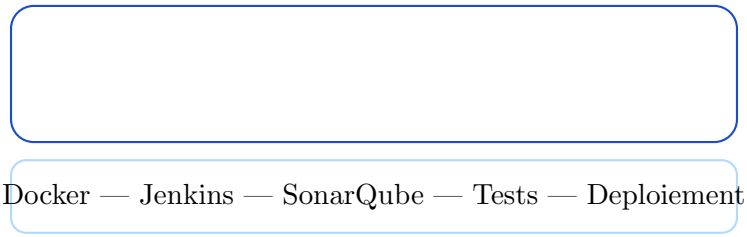
Travaux Pratiques 1

Mise en place d'une chaine CI/CD conteneurisee

avec controle qualite et DevSecOps



Docker — Jenkins — SonarQube — Tests — Deploiement

Outils utilises	Jenkins, Docker, SonarQube
Technologies	Python Flask, Docker, Kubernetes-ready
Format	Pipeline as Code (Jenkinsfile)
Securite	DevSecOps - Shift Left

Ce rapport explique l'architecture, les roles des stages et les apports de chaque composant du cycle DevOps

Table des matières

1 Introduction **2**

2 Architecture Globale **2**
2.1 Vue d'ensemble de la Pipeline . . . 2
2.2 Cycle DevOps Complet . . . 2

3 Les 7 Stages de la Pipeline **2**

4 Apports de Docker **3**
4.1 Reproductibilite . . . 3
4.2 Portabilite . . . 3
4.3 Isolation . . . 3
4.4 Scalabilite . . . 3

5 Apports de CI/CD **3**
5.1 Automatisation . . . 3
5.2 Detection Precoce . . . 3
5.3 Feedback Rapide . . . 3
5.4 Tracabilite Complète . . . 3

6 Controle Qualite (SonarQube) **3**
6.1 Securite . . . 3
6.2 Maintenabilite . . . 3
6.3 Quality Gates . . . 4
6.4 Shift Left - DevSecOps . . . 4

7 Comparaison : Avant vs Apres **4**

8 Exemple d'Execution **4**

9 Conclusions **4**
9.1 Points Cles . . . 4
9.2 Impact . . . 4

10 Livrables **5**
10.1 Fichiers Fournis . . . 5

1 Introduction

Ce projet demontre une approche DevOps **moderne et complete** pour une application Python Flask. La chaine CI/CD automatise l'ensemble du cycle de developpement, du commit au deploiement, en integrant :

— **CI (Continuous Integration)** : Build, Test, Analyse
— **CD (Continuous Deployment)** : Conteneurisation, Deploiement
— **DevSecOps** : Analyse de securite precoce (Shift Left)

2 Architecture Globale

2.1 Vue d'ensemble de la Pipeline

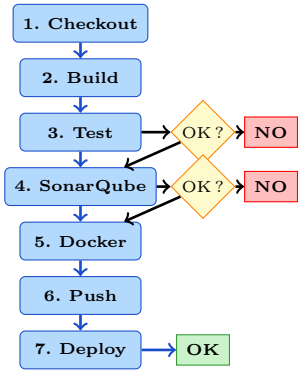


FIGURE 1 – Flux de la Pipeline CI/CD -

2.2 Cycle DevOps Complet

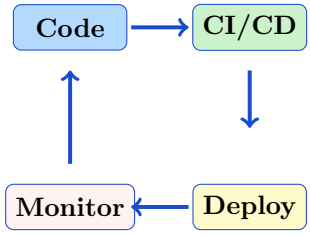


FIGURE 2 – Cycle DevOps Complet -

3 Les 7 Stages de la Pipeline

Stage 1 : Checkout

Role : Recuperer le code source depuis Git

Stage 2 : Build

Role : Installer les dependances Python

Stage 3 : Test

Role : Executer les tests unitaires
— Bloc si test echoue
— Detection precoce des bugs

Stage 4 : SonarQube

Role : Analyse statique et controle qualite
— Scan des vulnerabilites

— Detection du code duplique
— Quality Gates

Stage 5 : Docker Build

Role : Creer l'image Docker

Stage 6 : Push

Role : Envoyer l'image au registre

Stage 7 : Deploy

Role : Lancer l'application en production

4 Apports de Docker

4.1 Reproductibilite

Chaque deploiement utilise la meme image avec les memes dependances.

4.2 Portabilite

L'image Docker fonctionne sur Linux, Windows, macOS et le Cloud.

4.3 Isolation

Les conteneurs sont isoles les uns des autres sans conflits de dependances.

4.4 Scalabilite

Facile de deployer plusieurs instances rapidement.

5 Apports de CI/CD

5.1 Automatisation

Chaque push declenche automatiquement la pipeline.

5.2 Detection Precoce

Les bugs sont trouves en minutes, pas en jours.

5.3 Feedback Rapide

Les developpeurs recoivent une notification en quelques minutes.

5.4 Tracabilite Complète

Historique complet de tous les builds et deploiements.

6 Controle Qualite (SonarQube)

6.1 Securite

Detecte les vulnerabilites critiques avant le deploiement.

6.2 Maintenabilite

Reduit le code duplique et la dette technique.

6.3 Quality Gates

Bloc automatique si les standards ne sont pas respectes.

6.4 Shift Left - DevSecOps

La securite est integree tot dans le cycle, pas en production.

7 Comparaison : Avant vs Apres

Aspect	Sans CI/CD	Avec CI/CD
Deploiement	Manuel	Automatise
Tests	Optionnels	Obligatoires
Bugs detectes	2-3 jours	Quelques minutes
Reproductibilite	Problematique	Garantie
Securite	En production	Avant deploiement
Scalabilite	Difficile	Facile

TABLE 1 – Benefices du CI/CD

8 Exemple d'Execution

Listing 1 – Flux d'execution

```
$ git push origin main

[Stage 1] Checkout - OK
[Stage 2] Build - OK
[Stage 3] Test - FAIL (endpoint bug)
[STOP] Pipeline bloquee

Developpeur corrige le bug:

$ git push origin main

[Stage 1-4] - OK
[Stage 4] SonarQube - Vulnerabilites detectees
[Stage 5] Docker Build - OK
[Stage 6] Push - OK
[Stage 7] Deploy - OK

SUCCESS - Application en production
```

9 Conclusions

9.1 Points Cles

1. **Pipeline as Code** - Jenkinsfile versionne
2. **Automatisation Totale** - Zero intervention manuelle
3. **Reproductibilite** - Docker garantit la coherence
4. **Detection Precoce** - Bugs trouves en minutes
5. **Securite** - Shift Left avec SonarQube
6. **Traçabilite** - Logs complets de chaque etape

9.2 Impact

— **Developers** : Feedback rapide, tests auto, quality check
— **Operations** : Deploy facile, rollback possible, monitoring 24/7
— **Business** : Time-to-market reduit, quality augmentee, risque diminue

10 Livrables

10.1 Fichiers Fournis

— Dockerfile - Conteneurisation
— Jenkinsfile - Pipeline CI/CD (7 stages)
— app.py - Application Flask
— test_app.py - Tests unitaires (3 tests)
— requirements.txt - Dependances
— docker-compose.yml - Services optionnels
— README.md - Documentation complete
— ARCHITECTURE_PIPELINE.md - Diagrammes
— .gitignore - Configuration Git